PROJECT PROPOSAL

SwiftDrop

Real-Time Last-Mile Logistics Platform

Project Title	SwiftDrop — Real-Time Last-Mile Logistics Platform
Repository	github.com/SMAWAHID/Swiftdrop
Proposal Type	Full-Stack Web Application (Academic / Portfolio)
Core Technologies	FastAPI · PostgreSQL · React · TypeScript
Design Patterns	Singleton · Factory · Facade (GoF)
Primary Challenge	Concurrent order acceptance with row-level locking
Members	Syed Mawahid · Naheel Siddqui · Muhammad Sohaib
Roll No.	24K-1041 · 24K-0830 · 24K-0577

1. Problem Statement

Last-mile delivery is one of the most operationally complex segments of modern logistics. Existing platforms struggle with three core technical problems:

1. Race Conditions in Order Acceptance

When multiple drivers view and attempt to accept the same pending shipment simultaneously, naive implementations allow double-assignment — a single shipment is accepted by two drivers, causing operational chaos and poor customer experience.

2. Role Isolation & Access Control

Vendors, drivers, and administrators have fundamentally different views of the system. A single-dashboard approach leaks information across roles and creates UI complexity.

3. Scalable Shipment Lifecycle Management

Shipments pass through multiple states (PENDING → ACCEPTED → IN_TRANSIT → DELIVERED).
Managing this lifecycle cleanly across backend and frontend is non-trivial.

2. Proposed Solution

SwiftDrop addresses each problem with purpose-built engineering solutions:

Problem	Solution	Implementation
Race Conditions	Pessimistic Row-Level Locking	SELECT FOR UPDATE NOWAIT in PostgreSQL
Role Isolation	JWT Role Claims + Protected Routes	Role-based dashboards in React
Lifecycle Management	Facade Pattern + State Machine	ShipmentBookingFacade + status enum
Scalability	Async Python + Connection Pool	FastAPI + asyncpg Singleton pool
Auditability	PL/pgSQL Trigger + Session Var	SET LOCAL app.current_user_id per-request

3. Architecture Overview

SwiftDrop uses a strict layered N-Tier architecture separating HTTP handling, business logic, and data access.

Layer	Components	Responsibility
Presentation	React 18 + TypeScript + Vite	Role-differentiated dashboards, optimistic UI
Transport	FastAPI Routers	HTTP parsing, response serialization, exception mapping
Business	Service Classes (Facade, Factory)	Domain logic, validation, transactions
Data	asyncpg + PostgreSQL 15	Raw SQL, row-level locking, triggers, views

The Singleton pattern governs the asyncpg connection pool — initialized once at application startup via FastAPI's lifespan context and shared across all requests. The Factory pattern creates typed shipment objects (Standard, Express, Fragile) with different SLAs. The Facade pattern exposes the multi-step booking workflow as a single clean interface.

4. Key Features

Feature	Description	Stakeholder
Shipment Booking	Vendor books Standard/Express/Fragile shipments via Fa	adeVendor
Order Acceptance	Driver accepts PENDING orders; row lock prevents double	-assDrivergnment
Real-Time Dashboard	Live order feeds with optimistic UI updates	All roles
Admin Analytics	Pre-aggregated driver performance metrics via DB view	Admin
Notifications	Event-triggered in-app notifications per user	All roles
Driver Reviews	Post-delivery ratings stored per shipment	Vendor / Admin
Availability Toggle	Driver can mark themselves available/unavailable	Driver
JWT Auth	Secure login with role-based access on every endpoint	All roles

5. Design Patterns in Detail

Singleton — DB Pool

Location: `backend/db/connection.py`

A single asyncpg connection pool is created at startup and reused across all requests. This prevents connection exhaustion on a shared PostgreSQL instance and eliminates per-request connection overhead.

Factory — Shipment Types

Location: `backend/factories/shipment_factory.py`

The factory accepts a shipment type string and returns a fully-configured domain object (Standard, Express, or Fragile). Each type encapsulates its own SLA rules and handling requirements, decoupling type logic from the booking flow.

Facade — Booking Workflow

Location: `backend/services/shipment_service.py`

ShipmentBookingFacade wraps vendor lookup, factory instantiation, database insertion, and notification dispatch into a single call. Routers remain thin — they call the facade and return the result, unaware of the internal steps.

6. Concurrency Strategy

The most critical correctness requirement in SwiftDrop is that no shipment can be accepted by more than one driver. The implementation uses PostgreSQL's pessimistic row-level locking:

Aspect	Detail
SQL Statement	SELECT ... FOR UPDATE NOWAIT
On lock success	Transaction proceeds; shipment status updated to ACCEPTED
On lock failure	LockNotAvailable exception caught; 409 Conflict returned immediately
HTTP thread impact	None — NOWAIT prevents any blocking wait on the DB thread
Alternative considered	Optimistic locking (version column + retry) — rejected
Rejection reason	Losers always fail; retries waste compute with no benefit

7. Database Schema Overview

The schema is fully normalized to 3NF across the following core tables:

Table	Purpose	Key Columns
users	Base identity for all roles	id, email, password_hash, role, created_at
vendor_profiles	Vendor-specific data	user_id (FK), company_name, gst_number
driver_profiles	Driver-specific data	user_id (FK), vehicle_type, license_number, available
shipments	Core shipment entity	id, vendor_id, driver_id, type, status, timestamps
reviews	Post-delivery ratings	shipment_id, driver_id, rating, comment
notifications	Event messages	user_id, message, read, created_at
audit_log (trigger)	Change tracking	table_name, record_id, action, user_id, changed_at

A PostgreSQL view pre-aggregates driver performance data (total deliveries, average rating, acceptance rate) for the Admin dashboard, avoiding expensive per-request aggregation queries.

8. API Design

All endpoints are RESTful and documented via FastAPI's auto-generated OpenAPI interface at /docs.

Authentication uses Bearer JWT tokens on all protected routes.

Method	Endpoint	Role	Description
POST	/api/auth/login	Any	Authenticate, receive signed JWT
POST	/api/auth/signup	Any	Register new user with role profile
POST	/api/shipments/book	VENDOR	Book shipment (Facade pattern)
GET	/api/shipments	Any	List shipments filtered by role + status
GET	/api/shipments/search	Any	Full-text search across shipments
PUT	/api/shipments/{id}/accept	DRIVER	Accept order (row-level lock)
PUT	/api/shipments/{id}/complete	DRIVER	Mark shipment delivered
GET	/api/drivers/dashboard	ADMIN	Aggregated performance metrics
PATCH	/api/drivers/me/availability	DRIVER	Toggle driver availability
POST	/api/reviews	VENDOR	Submit post-delivery review
GET	/api/notifications	Any	Fetch user notifications

9. Frontend Architecture

Component / Module	Purpose
AuthContext.tsx	JWT state management via useReducer; persists token to localStorage
useShipment.ts	Custom hook for shipment data with optimistic UI updates on accept
ProtectedRoute (App.tsx)	Role-aware route guard; redirects unauthorized access
shipmentApi.ts	Axios repository pattern; all API calls centralized here
VendorDashboard	Booking form, shipment list with status tracking
DriverDashboard	Available orders feed, accept button, availability toggle
LoginPage	JWT login form with error handling and redirect
OrderCard / StatusBadge	Reusable shipment display components with color-coded status

10. Success Criteria

Functional Correctness: No shipment can be accepted by more than one driver under any concurrent load

Pattern Adherence: All three GoF patterns are demonstrably implemented and documented

Zero Frontend Errors: npm run build produces zero TypeScript errors

Role Isolation: Each role sees only its authorized data and actions
API Documentation: Full OpenAPI spec auto-generated and accessible at /docs

Auditability: All data changes are captured in the audit log with user attribution

11. Conclusion

SwiftDrop proposes a technically rigorous, production-deployable logistics platform that showcases advanced full-stack engineering. Its core contributions are a correct solution to the distributed order-acceptance race condition, a clean application of three GoF patterns in a real domain context, and a type-safe, role-differentiated user interface backed by a well-normalized PostgreSQL schema. The project is immediately runnable from the provided repository with a single SQL migration and two server start commands.

github.com/SMAWAHID/Swiftdrop